

NCAT（神经计算机自适应测试）精读笔记

原文：03296-ZhuangY.pdf (AAAI-22, Yan Zhuang 等, USTC) · 代码：<https://github.com/bigdata-ustc/NCAT>
(已下载到本目录 NCAT/) · 笔记于 2026-06-26

一、解决什么问题

场景背景。CAT（计算机自适应测试）想做的事：用尽量少的题目，准确测出一个学生的能力 θ 。它由两个组件交替工作：

- **CDM（认知诊断模型）**：根据学生已答的题，估计她现在的能力 $\hat{\theta}$ 。最经典的是 IRT。
- **选题算法（Selection Algorithm）**：根据 $\hat{\theta}$ 决定下一题考什么。这是 CAT 的核心，也是本文要改进的对象。

痛点。传统选题算法（MFI、KLI 等）是人手设计的、规则固定的，靠一些“信息量/不确定性”指标来挑題。它们有三个毛病：

1. 对学生：严重依赖当前 $\hat{\theta}$ 估得准不准；用单个标量 θ 概括“学生 × 题目”的复杂交互，信息损失大、不鲁棒。
2. 对 CDM：算法是“模型相关”的——换一个 CDM 就得重新为它量身设计一套选题规则。
3. 对题目：固定规则有选题偏好（比如 MFI 老爱选高分度的题），导致某些题被反复曝光，曝光率失衡、测试安全性下降。

本文主张。与其再手工设计一套更精巧的规则，不如把选题算法本身当成可学习的东西，直接从海量学生作答数据里学出来。这就是 NCAT：把 CAT 重新表述成强化学习问题，用神经网络当选题策略。

和已有工作比好在哪。最接近的前作是 BOBCAT（也用双层优化从数据学选题）。但 BOBCAT 没建模学生一题目交互的复杂性，也管不住曝光率。NCAT 用 RL + 注意力机制补上这两点。

二、问题定义（输入 / 输出）

符号。- 学生能力 $\theta \in \mathbb{R}^d$ ，真实值 θ_0 未知（ d 是知识概念数等维度）。- 题库 $\mathcal{J} = \{q_1, \dots, q_{|\mathcal{J}|}\}$ 。- 一条作答记录 (q, a) ，答对 $a = 1$ 、答错 $a = 0$ 。- 测试在第 t 步的状态： $s_t = \{q_1, a_i(1), \dots, q_{t-1}, a_i(t-1)\}$ ，就是“到目前为止答过的题及对错”。

任务一步步是这样的：第 t 步选题算法 π 看着当前估计 $\hat{\theta}^{t-1}$ 选一题 $q_t \sim \pi(\hat{\theta}^{t-1}) \rightarrow$ 学生作答得 $a_t \rightarrow$ CDM 更新出新的 $\hat{\theta}^t \rightarrow$ 重复 T 步（论文设 $T = 20$ ）。目标是测试结束时 $\hat{\theta}^T \rightarrow \theta_0$ 。

输入 / 输出（就 NCAT 这个选题网络而言）：- 输入：当前状态 s_t ——即学生已答的“答对题集合”和“答错题集合”。- 输出：对题库里每道题的一个分数 $Q_\pi(s_t, q)$ （动作价值），分数最高（或按概率采样）的那道题就是下一题。

一个具体小例子。题库有 5 道题 $\{q_1, \dots, q_5\}$ 。某学生已答 3 题： q_1 对、 q_3 错、 q_5 对。- 那么 $s_t = \{\text{答对集合 } \{q_1, q_5\}, \text{答错题集合 } \{q_3\}\}$ ($k_1 = 2$ 道对, $k_0 = 1$ 道错)。- NCAT 吃进这个状态，吐出 5 维向量，例如 $Q = [-\infty, 0.8, 0.2, -\infty, 0.5]$ （已答过的 q_1, q_3, q_5 被屏蔽成 $-\infty$ ，不能重复选）。- 取 $\arg \max \rightarrow$ 选 q_2 作为第 4 题。

代码印证：agents/Train.py 里 `policy = self.fa.predict(data)[0]`，然后 `for item in actions:`
`policy[item] = -np.inf`（屏蔽已答），不在候选集的也设 $-\infty$ ，最后 `action = np.argmax(policy[1:]) + 1`。索引 0 是 padding 位。

三、模型结构

整篇方法分两层来理解：(A) 怎么把“学选题算法”这件事形式化成一个可优化的目标；(B) 用什么网络去实现这个策略。

A. 从双层优化 $\rightarrow$ 强化学习

1. 双层优化（meta-learning 视角）。把每个学生的作答随机分成支持集 D_s^i （用来估能力）和查询集 D_u^i （用来评估选得好不好）。
 - 内层（式 3）：用 π 顺序选出的题 $D_s^{i(t)}$ ，在上面最小化交叉熵，估出 $\hat{\theta}_i^t$ 。
 - 外层（式 2）：在查询集 D_u^i 上看 $\hat{\theta}_i^t$ 拟合得好不好，用这个误差去优化 π 。
 - 直觉： θ_0 不知道，没法直接监督“估得准不准”，于是拿“在没考过的题（查询集）上预测得准不准”当代理信号。选的题目好 $\rightarrow$ 能力估得准 $\rightarrow$ 查询集上预测误差小。
2. 转成 RL（式 4）。把“最小化 $\sum_t$ 损失”等价改写成“最大化 $\sum_t$ 负损失”，负损失就成了奖励。于是 MDP 是：

- 状态 $s_t =$ 已答题及对错；动作 = 选哪道题（每题每人至多选一次）；
- 奖励 $r_i^t = -L_{\mathcal{M}}(D_u^i, \hat{\theta}_i^t)$ = 这一步选完题、更新能力后，在查询集上的负交叉熵；
- 用 Q-learning 学策略。
- 为什么要套 RL 而不直接贪心？因为选题是序列决策，“一组多样而互补的题”长期看比“每步贪心选最优”更能全面测准能力 $\rightarrow$ RL 能从长远视角探索更好的题目组合。

B. 分层注意力选题网络（论文图 2b，代码 functionApproximation/NCAT.py）

数据流：题目嵌入 $\rightarrow$ 双通道表现学习 (PL) $\rightarrow$ 矛盾学习 (CL) $\rightarrow$ 策略层 $\rightarrow$ Q 值。

- 题目嵌入：每道题学两套 embedding——答对用 E_1 、答错用 E_0 （同一题答对/答错携带的信息不同）。代码里就是 `self.q_embed_0` 和 `self.q_embed_1` 两个 `nn.Embedding`。
- 双通道表现学习 PL：把“答对的题”和“答错的题”分别送进各自的自注意力块编码（为什么分通道？因为答对/答错样本通常不均衡，答错的远少于答对的，混在一起会被淹没）。输出 F_t^0 （错通道）、 F_t^1 （对通道）。代码：`self.self_attn_0`、`self.self_attn_1`。
- 矛盾学习 CL：跨通道做 dual-attention，找“答对的难题 vs 答错的易题”这种矛盾，识别猜测/失误因子。输出矛盾特征 F_t^{01} 、 F_t^{10} 。代码：`self.contradiction = MultiHeadedAttention_con(...)`。
- 策略层：把四个矩阵 $\{F_t^0, F_t^1, F_t^{01}, F_t^{10}\}$ 各自平均池化后拼成 $u_t \in \mathbb{R}^{4d}$ ，过两层 MLP 输出每道题的 Q 值。代码：`self.policy_layer = nn.Sequential(nn.Linear(d_model*4, 512), ReLU, Dropout, nn.Linear(512, n_question))`，正好印证拼接维度是 $4d$ 、输出维度是题库大小。

四、关键公式详解（重点）

公式 (1): IRT —— 最基础的 CDM

$$P(a_{i,j} = 1) = \sigma[\alpha_j(\theta_i - \beta_j)]$$

- 符号： θ_i 学生 i 的能力（标量）； β_j 题 j 的难度； α_j 题 j 的区分度； σ 是 sigmoid。
- 举例：学生能力 $\theta = 1.0$ ，题难度 $\beta = 0.5$ 、区分度 $\alpha = 2$ 。则 $\alpha(\theta - \beta) = 2 \times 0.5 = 1.0$ ， $P = \sigma(1.0) \approx 0.73 \rightarrow$ 该生约 73% 概率答对。若换一道更难的题 $\beta = 1.5$ ： $2 \times (1.0 - 1.5) = -1.0$ ， $P = \sigma(-1.0) \approx 0.27$ 。能力一样，难题答对率就低，符合直觉。
- 作用：CDM 用它把“能力”翻译成“答对概率”，MFI 这类传统选题正是建立在这个公式的 Fisher 信息上。NCAT 不依赖它选题，但实验里用 IRT/NCDM 当被测的 CDM。

公式 (2)(3): 双层优化目标——“学选题算法”到底在优化什么

$$\pi^* = \arg \min_{\pi} \frac{1}{n} \sum_i \sum_t \frac{1}{|D_u^i|} \sum_{(q,a) \in D_u^i} l(a, \mathcal{M}(q|\hat{\theta}_i^t)) \quad \text{s.t.} \quad \hat{\theta}_i^t = \arg \min_{\theta_i} \sum_{(q,a) \in D_s^{i(t)}} l(a, \mathcal{M}(q|\theta_i))$$

- 符号： l 二元交叉熵； $D_s^{i(t)}$ 是 π 选出的前 t 题（支持集）； D_u^i 查询集； $\mathcal{M}$ 是 CDM。
- 举例：学生有 10 条作答，7 条进支持集、3 条进查询集。内层： π 从这 7 题里顺序选若干题，IRT 在上面拟合出 $\hat{\theta}$ ；外层：拿 $\hat{\theta}$ 去预测那 3 条查询题，若预测得准（交叉熵小）说明 π 选的题信息量大。反向把这个误差传给 π ，让它学会“选信息量大的题”。
- 作用：这是 NCAT 的灵魂——把“设计选题规则”变成“优化一个目标”，从而模型无关、可自动适配任意 CDM。

公式 (4): 把双层优化变成 RL 奖励

$$\min_{\pi} \sum_t L_{\mathcal{M}}(D_u^i, \hat{\theta}_i^t) \triangleq \max_{\pi} \mathbb{E}_{i \sim \pi} \left[\sum_t -L_{\mathcal{M}}(D_u^i, \hat{\theta}_i^t) \right]$$

- 关键转换：“最小化查询集损失”=“最大化负损失的累积期望”。负损失 $-L_{\mathcal{M}}$ 就被定义为每步奖励 r^t 。
- 举例：第 5 步选完题、更新能力后，查询集交叉熵 $L = 0.40 \rightarrow$ 奖励 $r^5 = -0.40$ ；第 6 步又选一题降到 $L = 0.35 \rightarrow r^6 = -0.35$ （损失越小奖励越大）。Q-learning 就去最大化这些奖励之和。
- 作用：搭起“双层优化 $\leftrightarrow$ 强化学习”的桥，使 Q-learning 能上场。

公式 (5): 矛盾分数——本文最有特色的设计

$$\alpha_{ij} = \frac{W_2^{0,c} \mathbf{q}_i^0 \cdot (W_2^{1,k} \mathbf{q}_j^1)^\top}{\sqrt{d}}$$

- 符号: $\mathbf{q}_i^0$ 第 i 道答错题的 embedding, $\mathbf{q}_j^1$ 第 j 道答对题的 embedding; $W_2^{0,c}, W_2^{1,k}$ 是把它们投到 query/key 空间的可学习矩阵; $\sqrt{d}$ 缩放。 α_{ij} 是“错题 i 与对题 j ”的矛盾分数, 所有 α_{ij} 拼成 $A \in \mathbb{R}^{k_0 \times k_1}$ 。
- 举例 (论文图 4a 的真实案例): 某生答对了较难的 q_{2335} (难度 0.171), 却答错了较简单的 q_{2348} (难度 0.151)。这俩知识点相关, 矛盾分数 α 就高 $\rightarrow$ 模型怀疑 q_{2335} 是猜对的、或 q_{2348} 是失误。反观 q_{2335} 与 q_{4804} 难度对比相似但知识点不相关, α 就低 $\rightarrow$ 不算矛盾, 忽略。
- 数值走一遍: 设 $d = 4$, 投影后错题向量 $W_2^{0,c} \mathbf{q}_i^0 = [1, 0, 1, 0]$, 对题向量 $W_2^{1,k} \mathbf{q}_j^1 = [1, 1, 1, 0]$, 点积 $= 1 \cdot 1 + 0 \cdot 1 + 1 \cdot 1 + 0 = 2$, 除以 $\sqrt{4} = 2 \rightarrow \alpha_{ij} = 1.0$ 。再对每行/每列分别 softmax 得 $\tilde{A}^0, \tilde{A}^1$, 用它们去加权对/错通道的特征 F_t^1, F_t^0 , 得到矛盾特征 F_t^{10}, F_t^{01} 。
- 作用: 显式建模猜测/失误因子, 把扰动从“真实能力信号”里剥离出来, 让选题更准。消融实验 (表 2) 里去掉它 (NCAT-P) 性能下降, 证明有用。
- 代码印证: MultiHeadedAttention_con.forward 里先 `scores, x1, attn_s = attention(query, key, value1)` 得到一个方向的注意力, 再 `x2 = torch.matmul(attn_s.transpose(-1,-2), value2)` 用转置的注意力得到另一个方向——正对应论文里 $\tilde{A}^0 F_t^1$ 和 $\tilde{A}_1^\top F_t^0$ 这一对。

公式 (6): 带温度的概率化选题——控曝光率

$$P(q|s_t) = \frac{e^{Q_\pi(s_t, q)/\nu}}{\sum_{q \in \mathcal{J}} e^{Q_\pi(s_t, q)/\nu}}$$

- 符号: ν 温度, 论文设 $\nu = 2^{-0.1t}$, 随步数 t 增大而减小。
- 举例: 三道候选题 Q 值 $[2, 1, 0]$ 。
 - 测试初期 $\nu = 1$: 概率 $\propto [e^2, e^1, e^0] = [7.39, 2.72, 1]$, 归一化 $\approx [0.67, 0.24, 0.09]$ ——有随机性, 第二、三题也可能被选, 路径更丰富。
 - 测试后期 $\nu = 0.1$: 概率 $\propto [e^{20}, e^{10}, e^0] \rightarrow$ 几乎 100% 选第一题, 趋近 $\arg \max$ 。
- 作用: 开头随机性大 $\rightarrow$ 不同学生从不同题开始 $\rightarrow$ 避免所有人都被喂同样的题 (曝光失衡)。表 3 显示默认 NCAT 曝光率仅 5.2%, 远低于 BOBCAT 的 16.3%。这是 NCAT 比 BOBCAT 多出的实用优势。

策略学习: Q-learning 损失

$$L(\varphi) = \mathbb{E}_{(s_t, q_t, r, s_{t+1}) \sim B} [(y_t - Q_\pi(s_t, q_t))^2], \quad y_t = r_i(s_t, q_t) + \gamma \max_{q_{t+1}} Q_\pi(s_{t+1}, q_{t+1})$$

- 标准 DQN: 目标值 y_t = 即时奖励 + 折扣后的下一步最大 Q 值 (Bellman), 最小化预测 Q 与 y_t 的均方差。 B 是回放缓冲区 (论文容量 10000), ϵ -贪心从 1 衰减到 0。
- 代码印证: NCAT.optimize_model 里 `loss_func = MSELoss, pre_value = policy[arange, target]` (取所选动作的 Q), `loss = loss_func(pre_value, goal)`; `Train.convert_batch2dict` 里 `goals = np.max(value, -1) * dones * gamma + goals` 正是在算 y_t 。

五、代码实现对照

下载到 NCAT/, 结构与论文对应如下:

论文内容	代码位置
选题网络 (PL+CL+ 策略层, 图 2b)	<code>functionApproximation/NCAT.py</code> $\rightarrow$ <code>class NCAT</code>
双通道自注意力 (PL)	<code>NCAT.self_attn_0 / self_attn_1</code> (Encoder+EncoderLayer, 标准 Transformer encoder)
矛盾学习 dual-attention (式 5)	<code>class MultiHeadedAttention_con</code> , 用 <code>attn_s</code> 和它的转置得两方向矛盾特征

论文内容	代码位置
策略层 $\rightarrow$ Q 值 (拼 $4d \rightarrow$ MLP)	NCAT.policy_layer, forward 里 torch.cat([input_0,input_1,input_01,input_10],-1)
Q-learning 训练循环 / ϵ -贪心 / 回放	agents/Train.py (MEMORYSIZE=50000、 decay_function 控探索与折扣)
屏蔽已答题、不重复选	Train.collecting_data_update_model: policy[item]=-np.inf
MDP 环境 (状态转移、奖励 = 查询集负损失)	envs/env.py、envs/pre_train/ (内含 IRT/MIRT/NCD 三个 CDM)
被测 CDM	envs/pre_train/CDM/IRT.py / MIRT.py / NCD.py

注意: 代码默认 torch.device('cuda'), 跑起来需要 GPU; 超参 ($d = 128$ 、lr=0.001、buffer=10000) 与论文一致。仓库自带的样例数据是 data/assist1213。

六、小结

- **贡献:** 第一个把 CAT 选题完整地表述成 **RL 问题并在真实数据上验证**的全自适应框架; 用 PL+CL 双注意力建模学生一题目复杂交互 (尤其猜测/失误因子); 用温度采样兼顾测量**准确性与曝光控制**。
- **效果:** 三数据集(ASSIST/EXAM/NIPS-EDU) $\times$ 两 CDM(IRT/NCDM)上, ACC、AUC 全面超过 MFI/KLI/MAAT/BOBCAT; 曝光率从 $\sim 16\%$ 降到 5.2% ; 在加了猜测/失误噪声后 MSE 仍稳。
- **优点:** 模型无关 (换 CDM 不用重设计)、长远视角选题更多样、可控曝光。
- **局限:**
 1. 动作空间 = 整个题库, 题库很大时策略层输出维度 $|\mathcal{J}|$ 会很大、训练更难;
 2. 沿用“测试期间能力不变”假设 (才用平均池化忽略顺序), 对带学习/遗忘的长程场景未必成立;
 3. 奖励依赖查询集划分, 对数据稀疏的学生估计可能不稳;
 4. 需要 GPU 与较大回放缓冲, 部署成本高于 MFI 这类无训练的规则法。